

TECHNICAL ARCHITECTURE

PROJECT SENTINEL

Autonomous AI-Powered Payment Risk Intelligence & Protection Platform

Combining Machine Learning, Graph Intelligence, Agentic AI and PayPal Payment Infrastructure

PREPARED FOR

PayPal AI Hackathon 2026

PREPARED BY

Tanisha Dey

DATE

October 2026

Disclaimer: Sentinel is a hackathon prototype. It runs on PayPal Sandbox with synthetic data only and is not an official, deployed or endorsed PayPal fraud-prevention product. It does not access PayPal's internal risk systems. Model results describe synthetic data and are not evidence of real-world performance.

About this document

An engineering-focused extract of the complete report: architecture, technology stack, ML pipeline, graph intelligence, investigation agent, PayPal integration, database and API, security, testing and limitations.

Item	Detail
Project	PROJECT SENTINEL: Autonomous AI-Powered Payment Risk Intelligence & Protection Platform
Prepared for	PayPal AI Hackathon 2026
Prepared by	Tanisha Dey
Source of truth	The repository (backend/, frontend/, docs/). Every metric and test result was taken from the running system or its test suite.
Repository	github.com/Tanisha-dey18/hackathon
Generated	2026-10-31

> **Disclaimer.** Sentinel is a hackathon prototype using PayPal Sandbox and synthetic data. It is not an officially deployed PayPal fraud prevention product and has no access to PayPal's internal systems. Features that are designed but not implemented are labelled **Future Scope**, and integrations that are implemented but were not executed live are labelled as such.

Contents

Part 5: System Architecture . . . 5

5.1 High-level architecture . . . 5
5.2 Frontend architecture . . . 5
5.3 Backend architecture . . . 6
5.4 Payment processing sequence . . . 7
5.5 Real-time communication . . . 7
5.6 Deployment architecture . . . 8

Part 6: Technology Stack Explained . . . 9

6.1 Frontend . . . 9
6.2 Backend, data and infrastructure . . . 9
6.3 Machine learning, graph and AI . . . 10
6.4 Payment . . . 10

Part 7: Machine Learning Deep Dive . . . 11

7.1 Dataset structure . . . 11
7.2 Pipeline overview . . . 12
7.3 Feature engineering (point-in-time) . . . 12
7.4 Preprocessing, split and class imbalance . . . 12
7.5 The four algorithms . . . 13
7.6 Model selection and evaluation . . . 13
7.7 Understanding the metrics . . . 14
7.8 Explainable AI with SHAP . . . 14
7.9 Risk scoring workflow . . . 15
7.10 Retraining and feedback . . . 15

Part 8: Graph Intelligence . . . 16

8.1 Why transaction-level analysis is not enough . . . 16
8.2 Architecture . . . 16
8.3 Detection algorithms . . . 17
8.4 What the graph found in the synthetic data . . . 17
8.5 How graph findings become evidence . . . 18

Part 9: Agentic AI Investigation System . . . 19

9.1 What "agentic" means here . . . 19
9.2 The workflow . . . 19
9.3 Tool calling with Groq . . . 20
9.4 Hallucination prevention . . . 20
9.5 Error handling and fallback . . . 20
9.6 Why the agent recommends but never acts . . . 21

9.7 End-to-end example (actual run, Demo C) . . . 21

Part 10: PayPal Integration . . . 22

10.1 Developer platform setup . . . 22

10.2 Calls Sentinel makes . . . 22

10.3 Payment lifecycle, step by step . . . 22

10.4 Two status fields, never mixed . . . 22

10.5 Idempotency and duplicate-capture prevention . . . 23

10.6 Webhooks . . . 23

10.7 Error handling . . . 23

10.8 Honest scope and verification status . . . 23

10.9 Why PayPal is central, not an add-on . . . 23

Part 11: Database and API Documentation . . . 24

11.1 Entity relationships . . . 24

11.2 Tables . . . 24

11.3 API endpoints . . . 25

11.4 Request and response examples . . . 27

Part 12: Security and Reliability . . . 28

Part 15: Testing and Results . . . 29

15.1 Automated test suite . . . 29

15.2 Other verification performed . . . 30

15.3 Not yet verified . . . 30

Part 17: Limitations and Future Scope . . . 31

17.1 Current limitations (implemented system) . . . 31

17.2 Future scope (not implemented) . . . 31

PART 5

System Architecture

5.1 High-level architecture

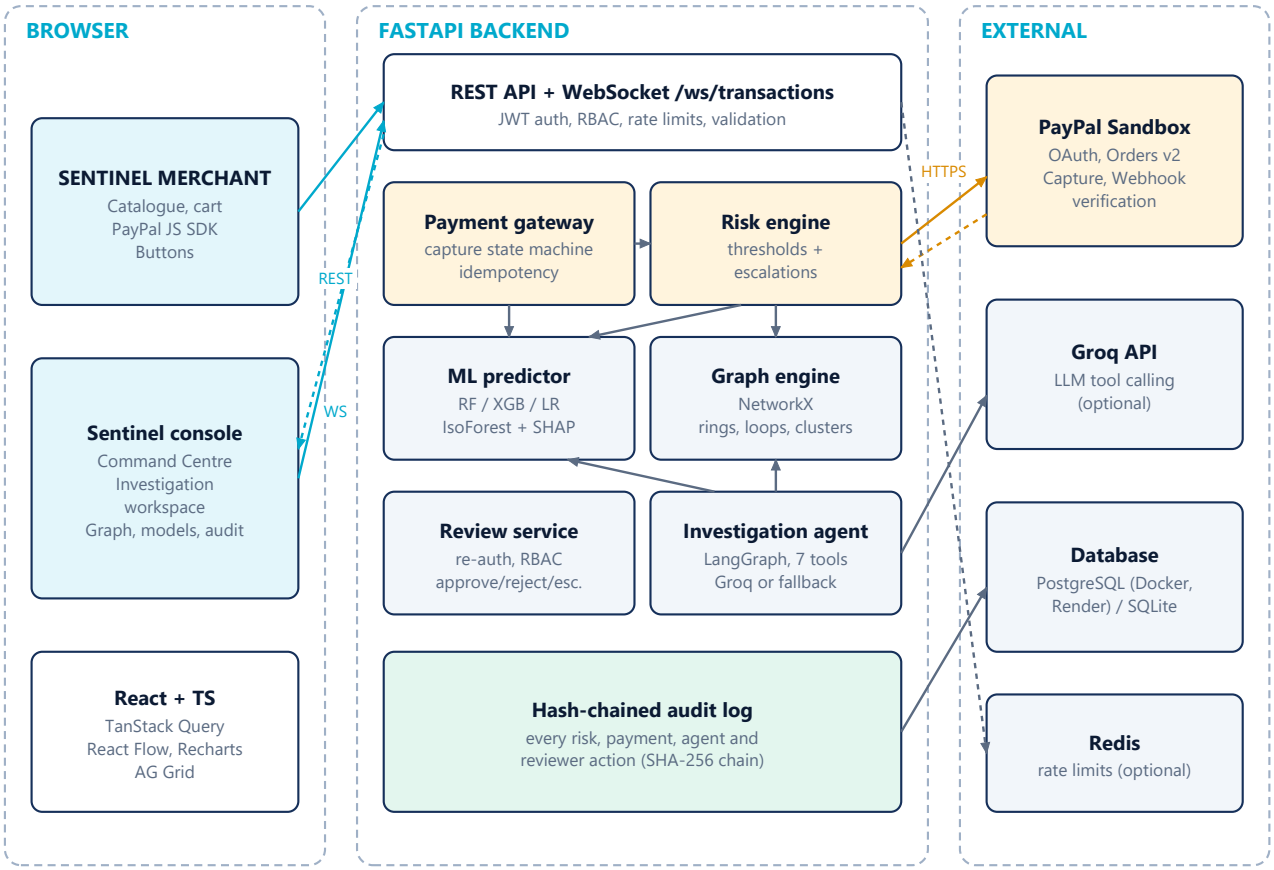


Figure 1: High-level system architecture

The system has three tiers. The **browser** runs a single React application with two areas: the SENTINEL MERCHANT storefront (shoppers) and the console (staff). The **FastAPI backend** exposes REST endpoints and one WebSocket, and contains the payment gateway, risk engine, ML predictor, graph engine, investigation agent and audit log. The **external services** are PayPal Sandbox (always used for real payments), Groq (optional, for the LLM agent), the database (PostgreSQL in Docker/Render, SQLite locally) and Redis (optional, for shared rate limiting).

Data flow in one sentence: the browser calls the API; the API stores state in the database, talks to PayPal for every payment operation, calls the risk engine (which uses the ML model and the graph), starts the agent for high-risk payments, records every step in the audit log, and pushes events back to the console over the WebSocket.

5.2 Frontend architecture

- **Routing** (`react-router-dom`): `/shop/*` for the storefront, `/console/*` for the console, `/login`.
- **State and data**: TanStack Query caches every API call. The `LiveProvider` opens one authenticated WebSocket and invalidates the relevant queries when events arrive, so every screen updates without manual refresh.
- **Auth**: `AuthProvider` stores the JWT in `sessionStorage` and exposes the current user and role. Console routes require a staff role.

- **Visualisation**: Recharts (dashboard and model charts), React Flow (relationship graph), AG Grid Community (transactions table), Framer Motion (transitions).
- **Payments**: `@paypal/react-paypal-js` renders PayPal Buttons using the public client ID returned by `GET /api/paypal/config`. The secret never reaches the browser.

5.3 Backend architecture

Module	Responsibility
`main.py`	FastAPI app, lifespan (migrations, first-run seeding and training, model artifact recovery), CORS, security headers, WebSocket.
`routers/`	`auth`, `transactions` (transactions, risk, reviews), `intel` (investigations, graph), `paypal` (checkout, webhook, storefront), `ops` (dashboard, audit, models, settings, demos).
`services/payments.py`	PayPal order lifecycle, capture state machine, webhook handling.
`services/risk.py`	Feature state, scoring, policy thresholds and escalations.
`services/review.py`	Reviewer decisions with re-authentication and RBAC.
`services/transactions.py`	Transaction creation and risk routing.
`ml/`	Feature engineering, training, SHAP explanation, prediction.
`graph/engine.py`	Graph persistence, NetworkX analysis, explorer views.
`agent/`	Seven investigation tools and the LangGraph workflow.
`audit.py`	Hash-chained audit log and chain verification.
`infra.py`	Rate limiter (Redis or memory) and WebSocket broadcaster.

5.4 Payment processing sequence

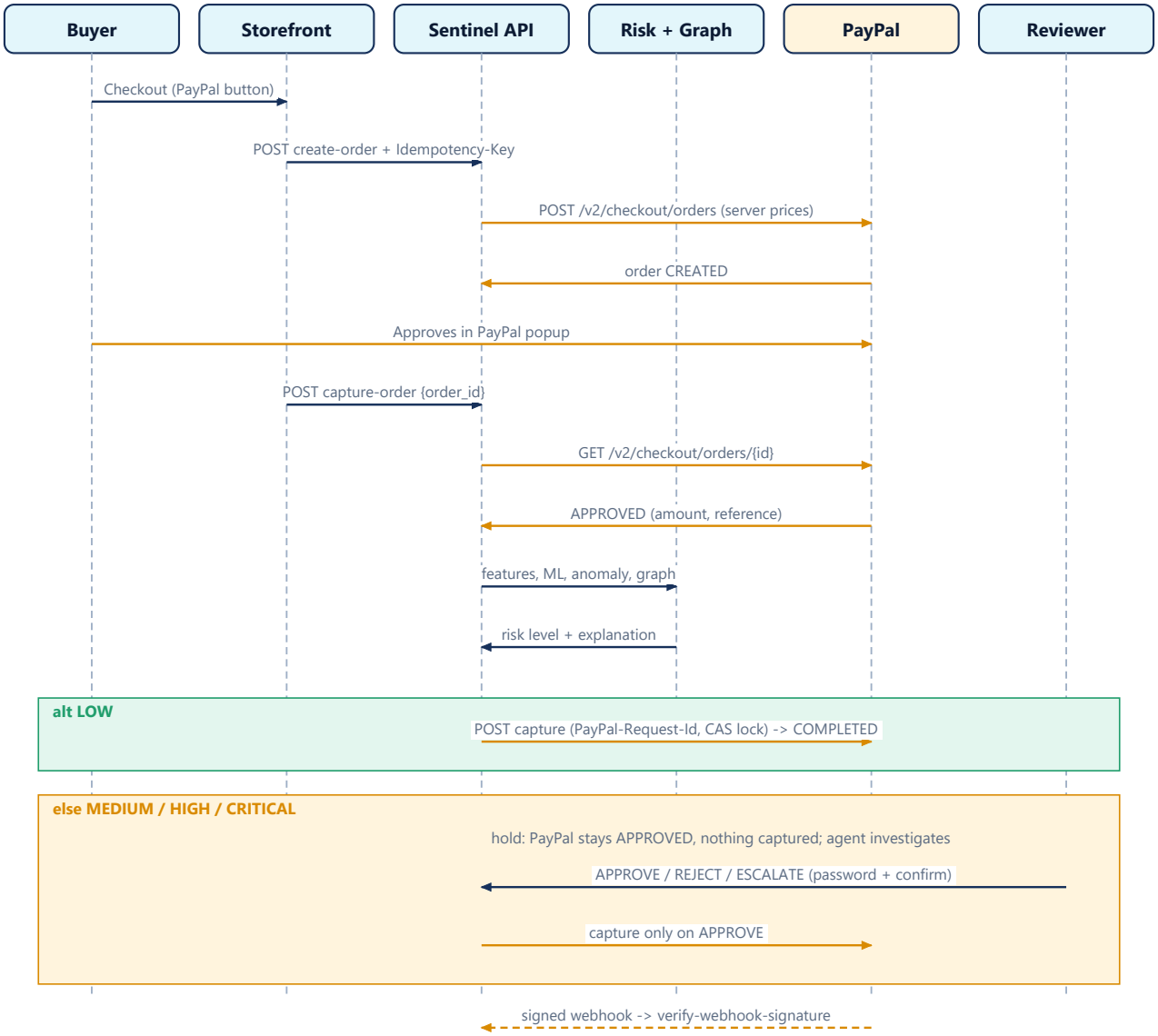


Figure 2: Payment processing sequence (Orders v2 with Sentinel risk gating)

The sequence shows the two independent approvals in the lifecycle: the buyer's approval in PayPal, and Sentinel's risk approval. The capture call to PayPal only happens if both exist. If the risk policy holds the payment, the reviewer's APPROVE decision replaces the automatic approval, and the same capture function is used, with the same protections.

5.5 Real-time communication

`ws://<host>/ws/transactions?token=<JWT>` accepts staff tokens only. The backend broadcasts `transaction.created`, `transaction.assessed`, `transaction.updated`, `payment.updated`, `investigation.step`, `investigation.completed` and `decision.made`. Because many events come from background threads (the agent, the simulator), the broadcaster uses `asyncio.run_coroutine_threadsafe` with the server's event loop.

5.6 Deployment architecture

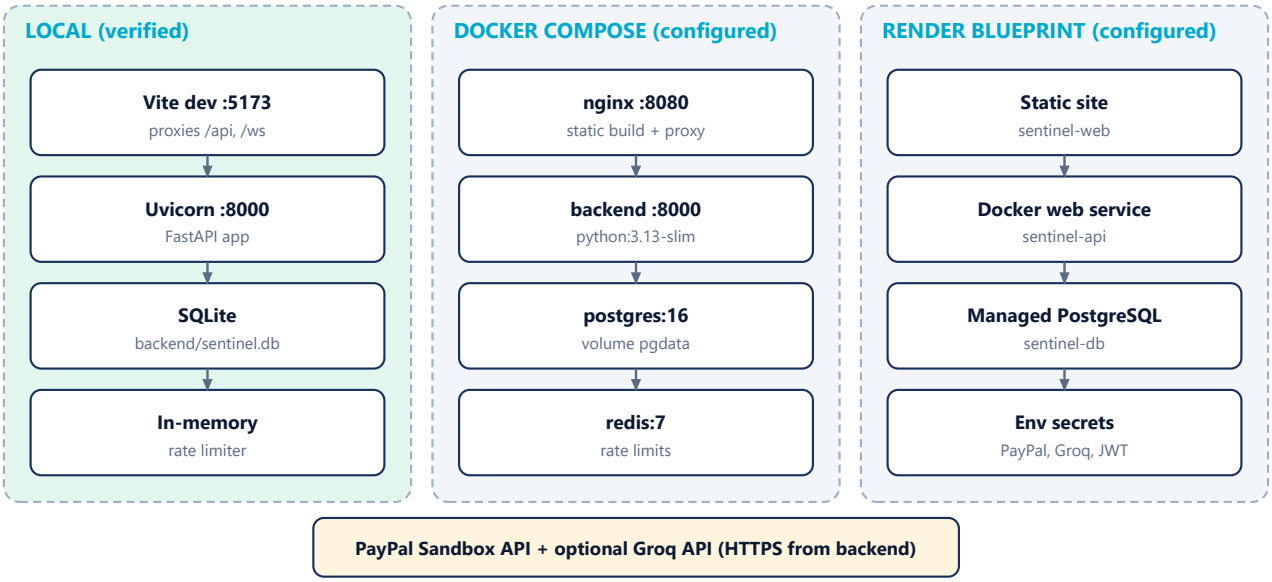


Figure 3: Deployment architecture

Three supported ways to run Sentinel:

Mode	Components	Status
Local	Uvicorn + Vite dev server, SQLite, in-memory rate limiting	Run and verified during development
Docker Compose	PostgreSQL 16, Redis 7, backend container, nginx serving the built frontend and proxying `/api` and `/ws`	Configuration written; not run during development because Docker was not running on the build machine
Render Blueprint	Managed PostgreSQL, Docker web service for the API, static site for the frontend	Configuration written; not deployed

On hosts with ephemeral disks, the backend retrains automatically at start-up if the active model file is missing.

PART 6

Technology Stack Explained

6.1 Frontend

Technology	What it is	Why chosen / problem solved	Where used	Talks to
React 18	UI component library	Component model for two applications in one bundle	All screens	TanStack Query, router
TypeScript 5.8	Typed JavaScript	Catches API shape mistakes at build time	Entire frontend	Compiler only
Vite 6	Build tool and dev server	Fast reloads; dev proxy for `/api` and `/ws`	Build and dev	Proxies to FastAPI
Tailwind CSS 3.4	Utility CSS framework	Consistent dark fintech design tokens	All styling	Build step
Framer Motion 12	Animation library	Smooth panel and list transitions	Cards, modals, feeds	React
Recharts 2	Chart library	Area, bar, line, pie, ROC/PR and SHAP charts	Command Centre, Fraud Intelligence, Model Analytics	API data via TanStack Query
React Flow (@xyflow/react 12)	Interactive node graph	Pan, zoom, hover and click on relationship graphs	Network Graph, Investigation workspace	`/api/graph/*`
AG Grid Community 33	Data grid	Filtering, sorting, quick search, pagination and CSV export over 2,000+ rows	Live Transactions	`/api/transactions`
TanStack Query 5	Server-state cache	Caching, polling, invalidation from WebSocket events	All data access	REST API
Lucide React	Icon set	Consistent icons	Everywhere	n/a
PayPal React SDK (@paypal/react-paypal-js 8)	Wrapper for PayPal JS SDK	Renders PayPal Buttons, handles approval popup	Checkout	PayPal + Sentinel API

AG Grid was evaluated as the brief asked: it adds real value for filtering and searching thousands of rows. Row grouping is an Enterprise feature, so risk "grouping" is done with filter chips instead.

6.2 Backend, data and infrastructure

Technology	What it is	Why chosen / problem solved	Where used
Python 3.13	Language	Same language for API, ML and graph code	Backend
FastAPI 0.115	Async web framework	Typed endpoints, automatic OpenAPI/Swagger, native WebSockets	`main.py`, `routers/`
Pydantic 2 / pydantic-settings	Validation and settings	Strict request validation; secrets loaded from environment	Request models, `config.py`
SQLAlchemy 2	ORM	One model layer for PostgreSQL and SQLite; atomic compare-and-set updates	`models.py`, services
Alembic 1.15	Migrations	Versioned schema changes	`alembic/versions/0001_initial_schema.py`
PostgreSQL 16	Relational database	Production database in Docker/Render	`docker-compose.yml`, `render.yaml`

Technology	What it is	Why chosen / problem solved	Where used
SQLite	Embedded database	Zero-setup local development	Default `DATABASE_URL`
Redis 7 (optional)	In-memory store	Shared rate-limit counters across instances; falls back to memory if absent	`infra.py`
httpx	HTTP client	PayPal REST calls; injectable transport for tests	`paypal/client.py`
PyJWT, bcrypt	Auth libraries	Signed tokens, salted password hashing	`security.py`
Docker, Docker Compose	Containers	Reproducible multi-service environment	`Dockerfile`s, compose
Render	Cloud platform	Blueprint deployment of DB, API and static site	`render.yaml`
WebSockets (FastAPI/Starlette)	Push channel	Live console updates	`/ws/transactions`

6.3 Machine learning, graph and AI

Technology	Where used	Role
pandas 3, NumPy 2	`ml/train.py`, `ml/predictor.py`	Feature tables, metrics computation
scikit-learn 1.9	`ml/train.py`	Logistic Regression, Random Forest, Isolation Forest, imputation, scaling, metrics, stratified split
XGBoost 3.4	`ml/train.py`	Gradient-boosted tree classifier
SHAP 0.54	`ml/explain.py`	TreeExplainer / LinearExplainer attributions per prediction and globally
joblib	`ml/train.py`	Model bundle persistence
NetworkX 3.7	`graph/engine.py`	Graph construction, connected components, `simple_cycles`, Kamada-Kawai layout
LangGraph 1.2	`agent/workflow.py`	State-machine orchestration of the investigation
Groq SDK 1.7	`agent/workflow.py`	LLM tool calling (default model `llama-3.3-70b-versatile`, configurable)

6.4 Payment

Component	Where used
PayPal JavaScript SDK (Buttons)	`frontend/src/pages/shop/Checkout.tsx`
OAuth 2.0 client credentials	`PayPalClient._access_token()`
Orders API v2: create, get, capture	`PayPalClient.create_order / get_order / capture_order`
Webhook signature verification API	`PayPalClient.verify_webhook()`
PayPal Sandbox	Default `PAYPAL_BASE_URL = https://api-m.sandbox.paypal.com`

PART 7

Machine Learning Deep Dive

7.1 Dataset structure

All data is **synthetic** and generated deterministically (seed 2026) by `backend/app/seed/synthetic.py`. No real people, cards or accounts are involved.

Item	Count	Notes
Customers	100	Home city and spending profile each; three are demo personas (Ava, Ben, Cara)
Merchants	25	5 in high-risk categories: gift cards, crypto on-ramp, electronics resale, gaming credits, prepaid mobile
Devices	150	Personal, secondary, shared household, attacker and emulator devices
Payment destinations	46	25 merchant payout accounts, 19 customer-owned wallets, 2 flagged "mule" wallets
Products	16	Storefront catalogue
Historical transactions	2,000	Spread over 90 days
Labelled fraud	142 (7.1%)	Ground-truth label `is_fraud` exists only for synthetic history

The generator creates labelled behaviour patterns, including deliberately hard cases:

Pattern (scenario tag)	Transactions	Label
NORMAL	1,800	Legitimate (3% random declines)
RING_DEVICE_FARM	60	Mostly fraud (92%): 9 accounts, shared emulator devices, night bursts, mule wallets
CIRCULAR_RING	24	Fraud: 4 accounts passing money in a loop
ACCOUNT_TAKEOVER	20	Fraud: new device, far-away location, 8 to 25 times normal spend
CARD_TESTING	26	Fraud: tiny failed attempts then a large purchase
FRIENDLY_FRAUD	16	Fraud that looks completely normal (label noise)
LEGIT_LARGE	22	Legitimate large purchases (hard negatives)
P2P_LEGIT	32	Legitimate wallet transfers

Friendly fraud and legitimate large purchases are included on purpose: they stop the models from reaching unrealistic perfect scores.

7.2 Pipeline overview



Graph signals (shared device, flagged destination, cycle) enter at the Policy step and can only raise risk.

Figure 4: Machine-learning fraud detection pipeline

7.3 Feature engineering (point-in-time)

A transaction's features are computed **only from transactions that happened before it**. `FeatureState` replays history in time order for training, and the same class is used for live scoring. This prevents "looking into the future", and it guarantees the model sees identical feature definitions in training and production (no train/serve skew).

Group	Features (26 total)
Amount	`amount`, `log_amount`
Frequency	`txn_count_1h`, `txn_count_24h`, `txn_count_7d`, `hours_since_prev`
Customer baseline	`cust_avg_amount`, `amount_to_avg_ratio`, `amount_zscore`
Merchant	`merchant_risk_score`, `merchant_high_risk_category`, `is_p2p_transfer`
Device	`device_distinct_customers`, `device_new_for_customer`, `customer_distinct_devices_30d`
Geography	`geo_distance_prev_km`, `geo_velocity_kmh`, `distance_from_home_km` (haversine)
Failures	`failed_count_24h`, `failed_count_7d`
Network	`destination_distinct_customers`, `destination_flagged`, `customer_flagged_dest_interactions`
Time	`hour_of_day`, `is_night`, `is_first_txn`

7.4 Preprocessing, split and class imbalance

- **Cleaning**: drop unlabelled rows, duplicate IDs and non-positive amounts; clip extreme values (speed at 5,000 km/h, ratio at 200, z-score to [-50, 200]).
- **Missing values**: the first transaction of a customer has no history, so some features are missing. They stay `NaN` and are filled with the training median inside each model pipeline (`SimpleImputer`), so the test set never influences imputation.
- **Split**: stratified 75/25 hold-out with `random_state=42`: 1,500 training rows and 500 test rows (36 fraud cases in the test set).

- **Class imbalance** (about 1 fraud per 13 legitimate): `class_weight="balanced"` for Logistic Regression, `class_weight="balanced_subsample"` for Random Forest, and `scale_pos_weight = negatives / positives` for XGBoost.

7.5 The four algorithms

Logistic Regression

In plain words: it gives each feature a weight, adds them up, and squashes the total into a probability between 0 and 1.

Technically: p(fraud | x) = 1 / (1 + e^-(w·x + b)). The weights are learned by minimising the log-loss with L2 regularisation (C = 0.5). Features are standardised first.

Advantages: fast, stable, easy to interpret. **Limitations:** it only draws straight decision boundaries, so interactions (for example "new device AND far away") must be engineered by hand. **Role in Sentinel:** a transparent baseline. In v1 it had the best ROC-AUC (0.952) but lower precision (0.744).

Random Forest

In plain words: it grows hundreds of decision trees, each on a random sample of the data and features, and lets them vote.

Technically: bagging of CART trees (300 trees, min_samples_leaf = 2). The predicted probability is the average of the trees' leaf class proportions. Randomisation reduces variance compared with a single tree.

Advantages: captures non-linear interactions, robust to outliers, needs little tuning. **Limitations:** larger model, probabilities are not calibrated, less extrapolation. **Role in Sentinel:** the **primary model in v1**, because it had the highest PR-AUC (0.897).

XGBoost

In plain words: it builds trees one after another, each new tree fixing the mistakes of the previous ones.

Technically: gradient boosting. Each tree is fitted to the gradient of the log-loss; leaf weights use second-order (Hessian) information; regularisation (`reg_lambda = 1.5`, depth 4, learning rate 0.07, 250 trees, row and column subsampling) controls overfitting. `scale_pos_weight` up-weights fraud cases.

Advantages: usually the strongest family on tabular data; handles missing values; fast. **Limitations:** more hyperparameters; can overfit small data. **Role in Sentinel:** trained and evaluated every time; it would be selected automatically if it had the best PR-AUC. In v1 it came third by PR-AUC (0.886), so it was not the primary model. This is reported honestly rather than forcing XGBoost to be primary.

Isolation Forest

In plain words: unusual points are easier to "isolate" with random cuts than normal ones.

Technically: an ensemble of random isolation trees; the anomaly score is based on the average path length needed to isolate a point (shorter path = more anomalous). It is trained **without labels**. Sentinel converts the raw score into a percentile of the training scores ("more anomalous than 95% of training transactions").

Advantages: detects new kinds of unusual behaviour that labels do not cover. **Limitations:** "unusual" is not the same as "fraudulent". **Role in Sentinel:** a second opinion used by the policy (anomaly percentile at or above 0.98 raises LOW to at least MEDIUM) and shown to reviewers.

7.6 Model selection and evaluation

The primary model is chosen automatically as the supervised model with the highest **PR-AUC** on the hold-out set. All numbers below are the stored results of model version v1 (trained on 1,500 rows, evaluated on 500 rows the

models never saw), at threshold 0.5 for the supervised models:

Model	Precision	Recall	F1	ROC-AUC	PR-AUC	TN	FP	FN	TP
Random Forest (primary)	0.941	0.889	0.914	0.939	0.897	462	2	4	32
Logistic Regression	0.744	0.889	0.810	0.952	0.893	453	11	4	32
XGBoost	0.914	0.889	0.901	0.951	0.886	461	3	4	32
Isolation Forest (unsupervised)	0.700	0.778	0.737	0.940	0.830	452	12	8	28

> These metrics describe performance on synthetic data generated by the project itself. They demonstrate that the pipeline is correct and evaluated honestly; they are not evidence of real-world fraud detection accuracy.

7.7 Understanding the metrics

Metric	Meaning	In fraud terms
Precision	TP / (TP + FP)	Of the payments we flagged, how many were really fraud. Low precision = annoyed genuine customers.
Recall	TP / (TP + FN)	Of all fraud, how much we caught. Low recall = fraud slips through.
F1	Harmonic mean of precision and recall	One number that balances both.
ROC-AUC	Probability a random fraud case is scored above a random legitimate one	Threshold-free ranking quality; can look optimistic when fraud is rare.
PR-AUC (average precision)	Area under the precision-recall curve	Focuses on the rare fraud class; the most informative single number under imbalance.
Confusion matrix	TN, FP, FN, TP counts	Shows exactly which mistakes are made.
False positive	Genuine payment flagged	Customer friction, manual review cost.
False negative	Fraud not flagged	Financial loss, chargebacks.

Why accuracy is not enough: with 7% fraud, a model that says "never fraud" is 93% accurate and catches nothing. That is why Sentinel reports precision, recall, F1, ROC-AUC and PR-AUC and selects by PR-AUC.

7.8 Explainable AI with SHAP

SHAP (SHapley Additive exPlanations) splits one prediction into contributions from each feature, based on Shapley values from cooperative game theory: each feature's contribution is its average marginal effect over all orderings of features. For each prediction, the contributions plus the model's baseline add up to the model output.

Sentinel uses `TreeExplainer` for tree models and `LinearExplainer` for Logistic Regression. The explanation space is reported with every prediction: Random Forest values are in probability space; XGBoost and Logistic Regression values are in log-odds.

For each payment, positive contributions are converted into sentences a reviewer can check, for example "*Amount is 43.4x the customer's historical average*" or "*Device has been used by 4 distinct accounts*". Globally, the mean absolute SHAP value on the test set ranks the features. The top ten for v1 are:

Rank	Feature	Mean abs SHAP
1	amount	0.0799
2	log_amount	0.0709
3	hour_of_day	0.0668
4	distance_from_home_km	0.0650

Rank	Feature	Mean abs SHAP
5	is_night	0.0428
6	hours_since_prev	0.0278
7	amount_to_avg_ratio	0.0209
8	geo_distance_prev_km	0.0183
9	destination_distinct_customers	0.0182
10	merchant_risk_score	0.0154

Note: `hour_of_day` and `is_night` rank highly because the synthetic fraud ring operates at night. That is a property of the synthetic generator and a known limitation (Part 17).

7.9 Risk scoring workflow

1. Compute features from history (`FeatureState.features`).
2. Primary model gives `fraud_probability`; Isolation Forest gives `anomaly_score` (percentile); SHAP gives contributions.
3. Classify: LOW below 0.30, MEDIUM below 0.70, HIGH below 0.90, otherwise CRITICAL (editable by ADMIN).
4. Escalations (they can only raise risk): anomaly percentile at least 0.98 gives at least MEDIUM; amount at least 10 times average gives at least MEDIUM; graph signals (flagged destination, device shared by 3 or more accounts, circular loop) give at least HIGH.
5. Store a `fraud_predictions` row with the probability, scores, level, action, top factors, SHAP values, features, policy notes and latency; write an audit entry; broadcast an event.

Recommended actions: LOW `AUTO_APPROVE`, MEDIUM `ADDITIONAL_VERIFICATION`, HIGH `MANUAL_REVIEW`, CRITICAL `MANUAL_REVIEW_AND_INVESTIGATE`.

7.10 Retraining and feedback

`POST /api/models/retrain` (ADMIN) creates a new model version. It uses ground-truth labels for synthetic history and reviewer decisions for live payments (REJECT = fraud, APPROVE = legitimate). Previous versions remain stored.

PART 8

Graph Intelligence

8.1 Why transaction-level analysis is not enough

A model scores one payment using that customer's history. A fraud ring is designed so that each payment looks normal. The evidence is in the **connections**: one device used by several accounts, several accounts paying the same wallet, money going round in a circle. Graph analysis makes those connections visible.

8.2 Architecture

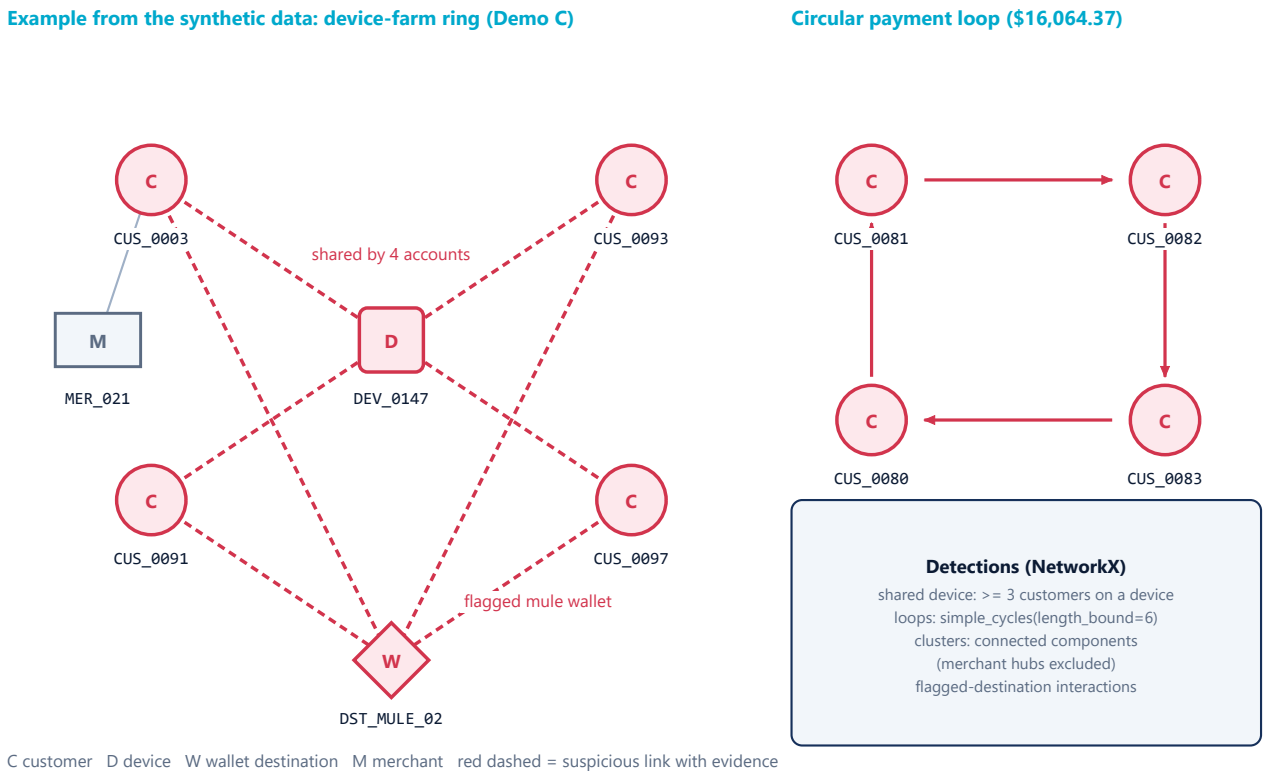


Figure 5: Graph intelligence: suspicious structures found in the synthetic data

Every transaction writes relationships into two tables: `graph_entities` (nodes) and `graph_relationships` (edges, with a weight counting repeats). The engine loads them into a NetworkX directed graph and then builds an **entity projection** that collapses transaction nodes into aggregated edges between customers, devices, destinations and merchants, each carrying a count, total amount and transaction IDs.

Node type	Example
CUSTOMER	CUS_0003 (Cara Novak)
MERCHANT	MER_021 (VaultGift Digital)
DEVICE	DEV_0147 (Android emulator)
DESTINATION	DST_MULE_02 (flagged external wallet)
TRANSACTION	TXN_100542

Edge type	Meaning
USES_DEVICE	customer used device

Edge type	Meaning
MADE_PAYMENT	customer made transaction
VIA_DEVICE	transaction made on device
PAID_TO	transaction paid destination
PAID_MERCHANT	transaction paid merchant
OWNS_WALLET	customer owns wallet destination

8.3 Detection algorithms

- **Shared devices:** device nodes connected to 3 or more customers.
- **Circular payments:** a directed customer-to-customer money-flow graph (payer to the owner of the wallet paid) searched with `networkx.simple_cycles(length_bound=6)`; loops of 3 or more accounts are reported.
- **Suspicious clusters:** connected components over customers, devices and wallets. Merchants and merchant payout accounts are excluded, otherwise everyone would be connected through popular shops. A cluster is suspicious if it contains a shared device, a flagged destination, a cycle member, or has mean account risk of at least 0.5.
- **Flagged destination interactions:** every account and payment that reached a flagged wallet.
- **Graph risk score:** a 0 to 1 score per customer from these signals (cycle 0.9, flagged destination 0.85, shared device 0.6 plus 0.05 per account up to 8, suspicious cluster 0.5 plus half the mean risk).
- **Traversal for views:** two-hop breadth-first expansion that never passes through merchant hubs; layout by Kamada-Kawai per connected component.

Community detection (for example Louvain) is **not implemented**; connected components are used. See Future Scope.

8.4 What the graph found in the synthetic data

Finding	Detail
Entities / relationships	2,326 / 8,114 (including demo transactions at time of writing)
Shared device DEV_0147	4 accounts: CUS_0003, CUS_0091, CUS_0093, CUS_0097
Shared device DEV_0148	3 accounts: CUS_0090, CUS_0094, CUS_0096
Circular loop	CUS_0081 to CUS_0082 to CUS_0083 to CUS_0080 and back, $16,064.37 in total
Flagged DST_MULE_01	16 payments, $11,238.59, from 7 accounts
Flagged DST_MULE_02	18 payments, $9,831.23, from 9 accounts
Suspicious clusters	4 (cluster 1: 9 accounts, 2 shared devices, 2 flagged destinations; cluster 8: the 4-account loop)

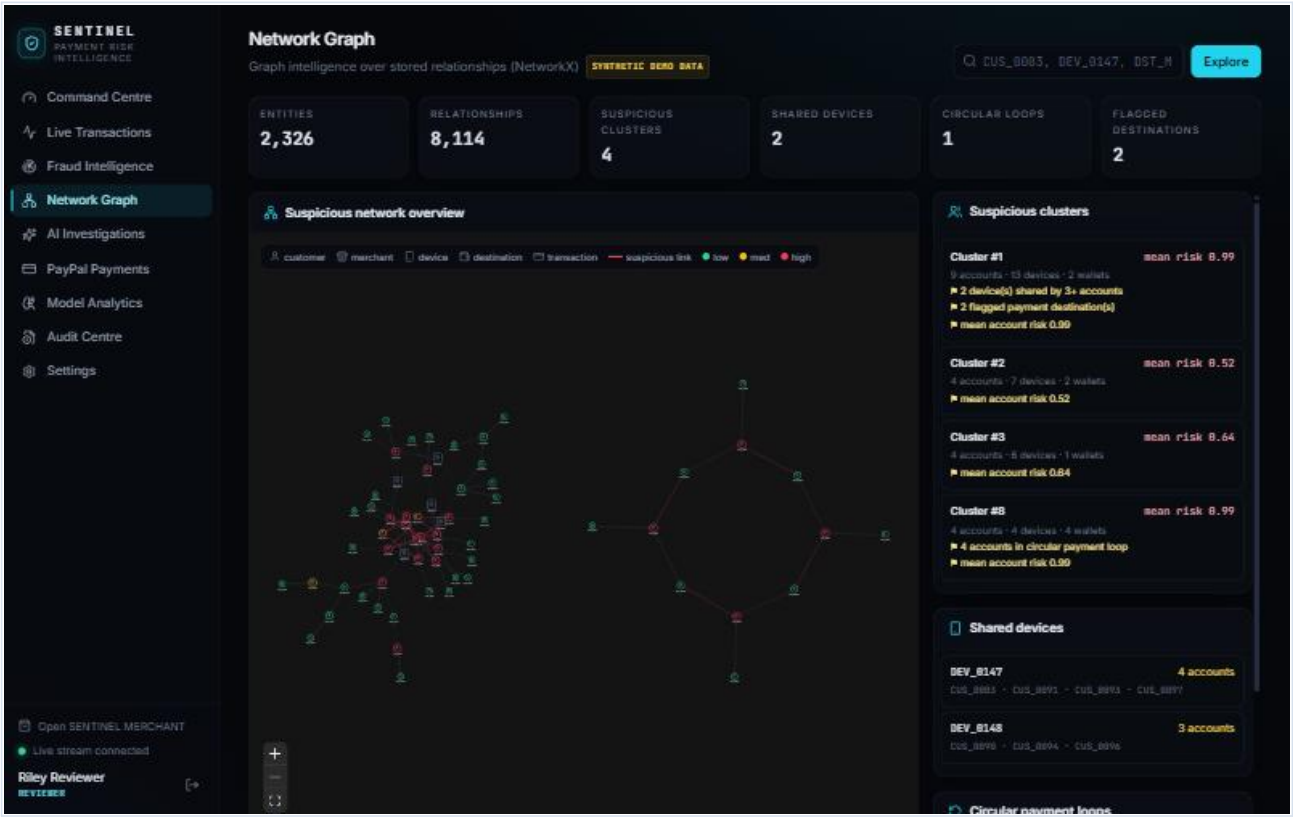


Figure 6: Network Graph overview: the device-farm cluster (left) and the four-account circular payment loop (right).

8.5 How graph findings become evidence

The `GraphInvestigationTool` returns the customer's signals; the `EvidenceCollectionTool` converts each into a typed evidence item such as "*E6 HIGH SHARED_DEVICE: Device DEV_0147 is shared by 4 accounts (CUS_0003, CUS_0091, CUS_0093, CUS_0097)*". Explorer edges carry the same evidence strings and transaction IDs, so a reviewer can click a red edge and open the underlying transactions.

PART 9

Agentic AI Investigation System

9.1 What "agentic" means here

Type	Behaviour
Traditional chatbot	Answers questions from its training; no access to your data.
Basic LLM application	Sends a prompt (perhaps with some data) and returns text.
Tool-using agent	The model decides to call functions (tools) that fetch real data, then uses the results.
Multi-step investigation workflow	A defined sequence of steps with shared state, tool calls, validation and a structured output.

Sentinel's investigator is the last kind: a LangGraph state machine whose fixed steps always run the core tools, followed by an analysis step in which an LLM may call more tools before submitting a structured report.

9.2 The workflow

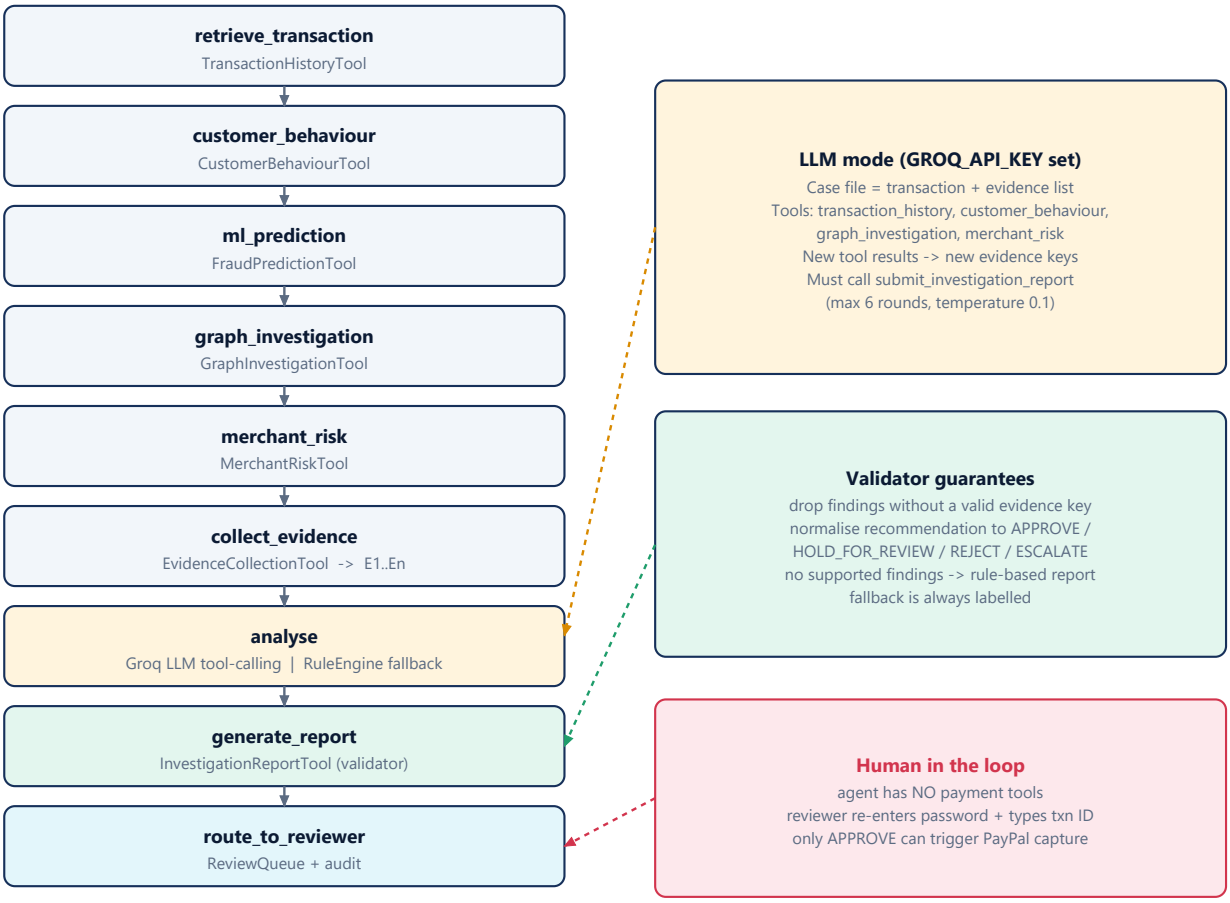


Figure 7: LangGraph investigation agent workflow

Node	Tool	Output
retrieve_transaction	TransactionHistoryTool	Focal transaction and up to 25 prior transactions
customer_behaviour	CustomerBehaviourTool	Average, median, devices, failures, distance from home, speed since previous

Node	Tool	Output
ml_prediction	FraudPredictionTool	Stored prediction, SHAP factors, policy notes (assesses first if none exists)
graph_investigation	GraphInvestigationTool	Shared devices, flagged destinations, cycles, cluster
merchant_risk	MerchantRiskTool	Merchant profile, or destination analysis for peer-to-peer transfers
collect_evidence	EvidenceCollectionTool	Evidence items E1…En with severity and "mitigating" flags
analyse	Groq LLM, or RuleEngine	Draft report
generate_report	InvestigationReportTool	Validated report; evidence stored in `investigation_evidence`
route_to_reviewer	ReviewQueue	Recommendation routed; audit entry

State management: a typed `InvState` dictionary carries the transaction ID, related entity IDs, tool outputs, draft, report and mode between nodes. Each step's start time, duration, status and summary is saved to `investigations.steps` and broadcast over the WebSocket, so the console shows progress live.

9.3 Tool calling with Groq

When `GROQ_API_KEY` is set, the analysis step sends the case file (transaction plus evidence list) to Groq with five tool definitions: `transaction_history`, `customer_behaviour`, `graph_investigation`, `merchant_risk` and `submit_investigation_report`. The model may call the first four to look at related entities (for example another account on the shared device). Their results become new evidence keys. It must finish by calling `submit_investigation_report` with a structured payload: `risk_level`, `summary`, `findings[{statement, evidence_keys}]`, `recommendation` (APPROVE, HOLD_FOR_REVIEW, REJECT or ESCALATE) and `rationale`. The loop is limited to 6 rounds, with temperature 0.1.

> Verification status: the Groq tool-calling path is implemented but was **not exercised** during development because no Groq API key was available. All investigations shown in this document used the deterministic fallback, and are labelled as such.

9.4 Hallucination prevention

1. Evidence is produced deterministically from tool outputs, never by the LLM.
2. The LLM only sees the case file and tool results.
3. `InvestigationReportTool` removes every finding that cites no valid evidence key and lists it under `rejected_findings`.
4. An invalid recommendation is replaced with `HOLD_FOR_REVIEW`.
5. If no supported findings remain, the rule-based report is used instead, with the reason recorded.
6. Tests check that invented keys (for example `E999`) and uncited claims are removed.

9.5 Error handling and fallback

If Groq is not configured, or the call fails, times out or does not submit a report within the step budget, the agent produces a rule-based report marked `DETERMINISTIC_FALLBACK` with the reason (for example "*GROQ_API_KEY not set*"). The UI shows "Deterministic rule-based report: not LLM-generated". A tool error marks that step as `ERROR` and the workflow continues; an unexpected exception marks the investigation `FAILED` with the error message.

9.6 Why the agent recommends but never acts

Capture, refund and rejection are irreversible financial actions, and LLMs can be wrong. The agent has **no payment tools**. Its only output is a recommendation placed in the review queue. A human with the REVIEWER or ADMIN role must decide, after re-entering their password and typing the transaction ID.

9.7 End-to-end example (actual run, Demo C)

Step	Duration	Result
Retrieve transaction	28.9 ms	CUS_0003 has 9 transactions; focal TXN_56SIN7VDWC $500.00
Analyse customer behaviour	12.7 ms	8 prior transactions, average $349.12, 2 devices, 0 failures
ML prediction + SHAP	11.5 ms	random_forest v1 scored 0.313 (final level HIGH); anomaly percentile 0.950
Graph connections	706.8 ms	1 shared device, 1 flagged destination link, suspicious cluster #1 (9 accounts)
Merchant risk	12.3 ms	VaultGift Digital: risk 0.72, chargeback rate 6.1%
Collect evidence	6.1 ms	9 evidence items, 4 high severity
Analysis	5.1 ms	Deterministic (Groq not configured)
Generate report	19.3 ms	REJECT, 6 evidence-backed findings
Route to reviewer	12.1 ms	Recommendation sent to reviewer queue

The graph step includes building the graph engine cache. The reviewer then rejected the payment with the reason "*Shared emulator device with 4 ring accounts and flagged mule destination (E6, E7).*"

PART 10

PayPal Integration

10.1 Developer platform setup

Item	How Sentinel uses it
PayPal Developer Dashboard	A Sandbox REST app is created under Apps & Credentials.
Sandbox environment	`PAYPAL_BASE_URL=https://api-m.sandbox.paypal.com`; no real money moves.
Sandbox business account	Owns the REST app and receives captured funds.
Sandbox personal account	Used by the tester to approve payments in the PayPal popup.
Client ID	Public; sent to the browser by `GET /api/paypal/config` to load the JS SDK.
Client secret	Server-only environment variable; never sent to the browser or logged.
Webhook ID	`PAYPAL_WEBHOOK_ID`, required to verify webhook signatures.

10.2 Calls Sentinel makes

Step	PayPal API	Sentinel code
OAuth token	`POST /v1/oauth2/token` (client credentials, cached until expiry)	`_access_token()`
Create order	`POST /v2/checkout/orders` with `intent=CAPTURE`, item breakdown, `reference_id/custom_id` = Sentinel transaction ID, `PayPal-Request-Id`	`create_order()`
Verify approved order	`GET /v2/checkout/orders/{id}`	`get_order()`
Capture	`POST /v2/checkout/orders/{id}/capture` with `PayPal-Request-Id: capture-{order_id}`	`capture_order()`
Verify webhook	`POST /v1/notifications/verify-webhook-signature`	`verify_webhook()`

10.3 Payment lifecycle, step by step

1. **PayPal order creation.** Sentinel creates the order; PayPal returns `status: CREATED`. Sentinel stores `paypal_status=CREATED`, `sentinel_status=AWAITING_BUYER_APPROVAL`.
2. **Customer approval.** The buyer approves in PayPal. PayPal's order becomes `APPROVED`. No funds have moved.
3. **Sentinel risk assessment.** On `capture-order`, Sentinel GETs the order, requires `APPROVED`, and checks amount, currency, reference and ownership. A mismatch sets `INTEGRITY_FAILED` and returns HTTP 409. Then it scores the risk.
4. **PayPal capture.** Only for LOW risk (automatic) or after a reviewer's APPROVE.
5. **Confirmed payment status.** Capture ID and capture status are copied from PayPal's response; `PAYMENT.CAPTURE.*` webhooks (when configured) update the same fields.

10.4 Two status fields, never mixed

Field	Written from	Example values
`paypal_status`, `capture_status`, `capture_id`	PayPal API responses and verified webhooks only	CREATED, APPROVED, COMPLETED; capture COMPLETED / PENDING / DENIED
`sentinel_status`	Sentinel's policy and reviewers	AWAITING_BUYER_APPROVAL, CLEARED, HELD_FOR_VERIFICATION, HELD_FOR_REVIEW, ESCALATED, APPROVED_AUTO, APPROVED_BY_REVIEWER, REJECTED, CAPTURE_FAILED, INTEGRITY_FAILED

A held payment therefore appears as PayPal `APPROVED`, capture "not captured", Sentinel `HELD_FOR_REVIEW`. The shopper sees: "*Payment approved by buyer but held by Sentinel for human review. Funds have NOT been captured.*"

10.5 Idempotency and duplicate-capture prevention

- **Order creation:** the client sends an `Idempotency-Key`; it is stored as a unique column and forwarded as `PayPal-Request-Id`. A repeated request returns the existing order.
- **Capture lock:** `capture_state` moves from NONE or FAILED to IN_PROGRESS with an atomic `UPDATE ... WHERE capture_state IN ('NONE','FAILED')`. Only the request that changes the row calls PayPal; any other returns "already performed or in progress".
- **PayPal-side idempotency:** a stable `PayPal-Request-Id = capture-{order_id}` means retries cannot capture twice at PayPal either.
- **Reconciliation:** if PayPal answers `ORDER_ALREADY_CAPTURED`, Sentinel GETs the order and copies the real capture.
- **Authorisation re-check:** the capture function itself requires a LOW assessment (automatic) or a stored APPROVE decision (reviewer).

10.6 Webhooks

Webhooks are verified with PayPal's verification API using `PAYPAL_WEBHOOK_ID`. Unverified events return HTTP 400 and are audited as `WEBHOOK_REJECTED`; duplicates (same event ID) are ignored. Handled events: `CHECKOUT.ORDER.APPROVED`, `PAYMENT.CAPTURE.COMPLETED`, `.DENIED`, `.PENDING`, `.REFUNDED`, `.REVERSED`. Webhooks need a public HTTPS URL, so they were not received during local development.

10.7 Error handling

PayPal errors raise `PayPalError` with HTTP status, issue code and `paypal-debug-id`. The error body is stored in `payment_events` and returned to the client as HTTP 502 with the debug ID. If credentials are missing, payment endpoints return HTTP 503 and the checkout screen explains how to configure PayPal; Sentinel never shows a simulated success.

10.8 Honest scope and verification status

- Sentinel **does not** access PayPal's internal fraud systems. It is a merchant-side risk layer that uses public PayPal REST APIs.
- The integration was tested end to end against a **mock PayPal transport** (a test double emulating Sandbox semantics, used only in tests).
- **Live Sandbox calls were not executed during development**, because Sandbox credentials were not available on the build machine. Configuring `PAYPAL_CLIENT_ID` and `PAYPAL_CLIENT_SECRET` and running Demo A and Demo C through checkout is the remaining verification step.

10.9 Why PayPal is central, not an add-on

The project's core guarantee, "risky money does not move without a human", only means something if Sentinel controls a real capture step. PayPal's order and capture separation is what makes risk gating possible, and PayPal's request IDs and webhook signatures are part of the safety design.

PART 11

Database and API Documentation

11.1 Entity relationships

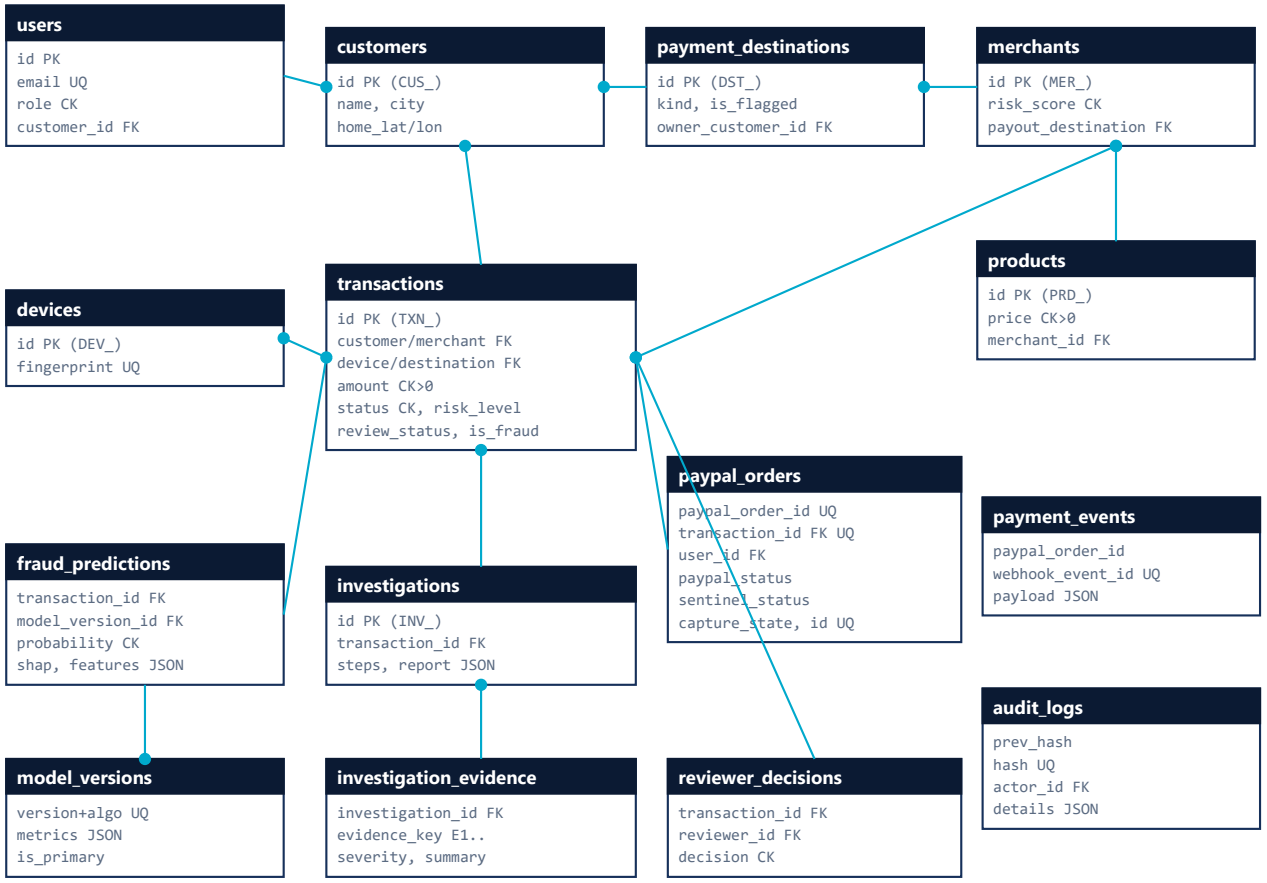


Figure 8: Database relationships (main tables and foreign keys)

11.2 Tables

All tables are created by Alembic migration `0001_initial_schema`. Timestamps are timezone-aware.

Table	Purpose	Key fields	Foreign keys	Constraints
users	Staff and shopper accounts	id PK, email (unique), password_hash, role, is_active	customer_id to customers	role in ADMIN, REVIEWER, ANALYST, CUSTOMER
customers	Synthetic customers	id PK (CUS_...), name, city, country, home_lat/lon, segment	none	none
merchants	Sellers	id PK (MER_...), category, risk_score, high_risk_category, chargeback_rate	payout_destination_id to payment_destinations	0 <= risk_score <= 1
devices	Device fingerprints	id PK (DEV_...), fingerprint (unique), type, os, is_flagged	none	unique fingerprint

Table	Purpose	Key fields	Foreign keys	Constraints
payment_destinations	Where money lands	id PK (DST_...), kind, is_flagged, flag_reason	owner_customer_id to customers	none
products	Storefront catalogue	id PK (PRD_...), name, price, stock	merchant_id to merchants	price > 0
transactions	Every payment	id PK (TXN_...), amount, lat/lon, channel, status, risk_level, review_status, is_fraud	customer, merchant, device, destination	amount > 0; status in allowed list; indexes on customer+time, device+time
fraud_predictions	Every risk assessment	probability, anomaly_score, graph_risk_score, risk_level, action, top factors, SHAP, features, policy notes, latency	transaction_id, model_version_id	0 <= probability <= 1
model_versions	Trained models	version, algorithm, is_active, is_primary, metrics JSON, artifact_path	none	unique (version, algorithm)
investigations	Agent runs	id PK (INV_...), status, mode, steps, report, recommendation	transaction_id, started_by_id	none
investigation_evidence	Evidence items	evidence_key, tool_name, category, severity, summary, data	investigation_id	none
graph_entities	Graph nodes	entity_type, ref_id, label, risk_score, flagged	none	unique (entity_type, ref_id)
graph_relationships	Graph edges	rel_type, weight	source_id, target_id to graph_entities	unique (source, target, type)
paypal_orders	PayPal order mirror	paypal_order_id (unique), amount, items, create_idempotency_key (unique), paypal_status, sentinel_status, capture_state, capture_id (unique), capture_status	transaction_id (unique), user_id	amount > 0
payment_events	PayPal API responses and webhooks	source, event_type, webhook_event_id (unique), signature_verified, http_status, paypal_debug_id, payload	none (indexed paypal_order_id)	unique webhook_event_id
reviewer_decisions	Human decisions	decision, reason, risk_level_at_decision, payment_outcome	transaction, investigation, reviewer	decision in APPROVE, REJECT, ESCALATE
audit_logs	Hash-chained audit trail	actor_label, action, entity, details, prev_hash, hash (unique)	actor_id to users	unique hash
system_settings	Key-value settings (risk policy)	key PK, value JSON	none	none

No card numbers, bank details or PayPal credentials are stored. Payer email and payer ID are stored only if PayPal returns them.

11.3 API endpoints

Authentication: `Authorization: Bearer <JWT>` from `POST /api/auth/login`. "Staff" means ADMIN, REVIEWER or ANALYST. Interactive documentation is at `/docs` (Swagger) and `/redoc`.

Method	Route	Purpose	Auth
POST	/api/auth/login	Email + password to JWT (rate limit 10/min)	Public

Method	Route	Purpose	Auth
GET	/api/auth/me	Current user	Any user
POST	/api/transactions	Create a synthetic transaction and assess it	Staff
GET	/api/transactions	List with filters: risk_level, status, channel, review_status, customer_id, q, limit, offset	Staff
GET	/api/transactions/{id}	Full case: customer, merchant, device, destination, predictions, investigations, decisions, PayPal order, events, audit	Staff
POST	/api/risk/assess	Re-assess a transaction with the active model	Staff
GET	/api/risk/metrics	Active model metrics and risk distribution	Staff
POST	/api/investigations/start	Start the agent (asynchronous)	Staff
GET	/api/investigations	List investigations	Staff
GET	/api/investigations/{id}	Steps, report, evidence	Staff
GET	/api/graph/overview	Suspicious clusters, shared devices, cycles, flagged destinations and graph	Staff
GET	/api/graph/entity/{id}	Neighbourhood of an entity (hops 1-3)	Staff
GET	/api/graph/transaction/{id}	Neighbourhood of a transaction	Staff
GET	/api/paypal/config	Public client ID and status (no secrets)	Public
POST	/api/paypal/create-order	Create PayPal order; header `Idempotency-Key` required	Shopper
POST	/api/paypal/capture-order	Verify, assess, then capture or hold	Order owner
GET	/api/paypal/order/{id}	Refresh state from PayPal	Owner or staff
GET	/api/paypal/orders	Orders (own for shoppers, all for staff)	Any user
GET	/api/paypal/events	PayPal API responses and webhooks	Staff
POST	/api/paypal/webhook	Verified PayPal webhooks (rate limit 300/min)	PayPal signature
POST	/api/reviews/{txn}/decision	APPROVE / REJECT / ESCALATE with password and confirmation	Reviewer, Admin
GET	/api/reviews/queue	Payments awaiting a decision	Staff
GET	/api/dashboard/metrics	KPIs, time series, distributions, network and model summary	Staff
GET	/api/audit	Audit log with filters	Staff
GET	/api/audit/verify	Verify the hash chain	Staff
GET	/api/models	All model versions and metrics	Staff
GET / POST	/api/models/retrain	Retrain status / start retraining	Staff / Admin
GET / PUT	/api/settings/risk-policy	Read / update thresholds	Staff / Admin
GET	/api/settings/integrations	PayPal, Groq, database, rate-limiter status	Staff
GET	/api/demo/scenarios	Demo scenario definitions	Staff
POST	/api/demo/scenarios/{A,B,C}/run	Run a scenario through the risk pipeline (no PayPal order)	Staff
GET / POST	/api/demo/simulator	Simulator status / start or stop	Staff
GET	/api/shop/products, /api/shop/products/{id}, /api/shop/demo-presets	Storefront data	Public
GET	/api/health	Liveness and integration flags	Public
WS	/ws/transactions?token=	Live events	Staff token

11.4 Request and response examples

Create order (`POST /api/paypal/create-order`, header `Idempotency-Key: 3f6c...`):

```
{"items": [{"product_id": "PRD_005", "quantity": 2}],
 "device_fingerprint": "fp-ava-macbook-air",
 "geo": null}
```

Response (shape from `order_view`; values from the test suite, which uses the mock PayPal transport):

```
{"paypal_order_id": "5O190100001TEST", "transaction_id": "TXN_...",
 "amount": 96.0, "currency": "USD", "paypal_status": "CREATED",
 "sentinel_status": "AWAITING_BUYER_APPROVAL", "capture_state": "NONE",
 "capture_id": null, "capture_status": null, "risk_level": null, ...}
```

Structured risk result (`POST /api/risk/assess`, abridged):

```
{"transaction_id": "TXN_56R036JSE2", "fraud_probability": 0.0983,
 "anomaly_score": 0.9547, "risk_level": "MEDIUM", "model_version": "v1",
 "model_algorithm": "random_forest",
 "top_risk_factors": ["Transaction amount $1,849.00",
   "Amount is 43.4x the customer's historical average",
   "First time this customer has used this device"],
 "recommended_action": "ADDITIONAL_VERIFICATION",
 "policy_notes": ["Model score 0.098 (random_forest v1) -> LOW",
   "Amount is 43.4x customer average (>= 10.0x) -> at least MEDIUM"]}
```

Reviewer decision (`POST /api/reviews/{txn}/decision`):

```
{"decision": "REJECT",
 "reason": "Shared emulator device with 4 ring accounts and flagged mule destination (E6, E7).",
 "password": "********", "confirm_transaction_id": "TXN_56SIN7VDWC"}
```

Common errors: 401 not authenticated, invalid token or re-authentication failed; 403 wrong role or not the order owner; 404 not found; 409 wrong state (for example not approved by buyer, already decided, amount mismatch, policy violation); 422 validation error (structured `errors` list); 429 rate limit; 502 PayPal error with `debug_id`; 503 PayPal not configured.

PART 12

Security and Reliability

Area	Implementation
Environment variables	All secrets come from environment / `.env` (`pydantic-settings`); `.env` is git-ignored; `.env.example` has blank values; blank values fall back to safe defaults.
Credential management	PayPal secret is used only server-side for OAuth; the client ID is the only PayPal value sent to the browser.
Authentication	bcrypt password hashes; HS256 JWT with expiry (default 480 minutes); failed logins audited.
Role-based access	FastAPI dependencies `require_staff`, `require_reviewer`, `require_admin`; shoppers cannot reach staff endpoints; staff cannot create checkouts.
Step-up for decisions	Password re-entry, typed transaction-ID confirmation, at least 20-character justification to approve HIGH/CRITICAL, ADMIN required for escalated cases.
Payment idempotency	Idempotency-Key and PayPal-Request-Id; compare-and-set capture lock; reconciliation of already-captured orders.
Payment ownership	Every order endpoint checks that the caller owns the order (or is staff where allowed).
Webhook verification	PayPal verification API with `PAYPAL_WEBHOOK_ID`; duplicate event IDs ignored.
Input validation	Pydantic models with ID patterns (`^TXN_\w+$` etc.), ranges and lengths; server-side pricing; quantity 1-10; single-seller carts; 256 KB webhook limit.
Rate limiting	Redis-backed when configured, otherwise in-memory; stricter limits on login, create-order and capture.
Audit logging	Every row stores SHA-256(previous hash + content); `/api/audit/verify` detects modification; tested by tampering a row.
Database integrity	Foreign keys, check constraints, unique keys, indexes; SQLite runs with `PRAGMA foreign_keys=ON`.
Error handling	Generic 500 responses; structured 422 errors; PayPal debug IDs preserved.
Secure logging	A logging filter redacts bearer tokens, client secrets and access tokens.
HTTP headers	`X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer`; explicit CORS allow-list.
Synthetic data privacy	All people, merchants and devices are generated; device fingerprints are random per-browser IDs.

Current security limitations: JWT is stored in `sessionStorage` (vulnerable if the page has an XSS flaw; an httpOnly cookie would be stronger); no multi-factor authentication; no formal security review or penetration test; the demo password is shared and published for judging; there are no rate-limit or WebSocket protections for distributed deployments beyond Redis counters.

PART 15

Testing and Results

15.1 Automated test suite

The backend has 23 automated tests (`pytest`) in `backend/tests`. They run against an isolated SQLite database that is seeded, trained and scored from scratch, and a **mock PayPal transport** that emulates Sandbox behaviour (OAuth, Orders v2, capture with `PayPal-Request-Id`, `ORDER_ALREADY_CAPTURED`, webhook verification). The mock is only used by tests.

Latest run: **23 passed, 0 failed (17.75 s)**.

Test	Area	Checks
test_models_trained_with_honest_metrics	ML evaluation	All 4 models trained; metrics in [0,1]; confusion matrix sums to test rows; primary has best PR-AUC
test_ml_prediction_structure	ML prediction	Probability and anomaly in [0,1]; SHAP for every feature; positive top factors; missing values kept as null
test_risk_classification_policy	Risk classification	Threshold mapping; escalation never lowers risk
test_transaction_creation_and_assessment	API	401 unauthenticated, 403 shopper; risk payload; CLEARED status; audit entry; 422 on negative amount
test_anomalous_amount_escalates	Risk policy	Demo B is MEDIUM or higher and held; amount factor present
test_graph_detects_fraud_structures	Graph	DEV_0147 shared; 4-account cycle; both mule wallets; clean customer has no signals; evidence on edges; 404 unknown entity
test_agent_tools_return_real_records	AI tools	Tools return real records; ToolError on unknown IDs; report tool drops invented and uncited findings; invalid recommendation normalised
test_investigation_workflow_deterministic_fallback	Agent workflow	All 7 tools run; labelled fallback; findings cite real keys; agent does not move money
test_rbac_and_auth	Security	401 / 403 rules; forged token rejected; admin-only policy update
test_audit_chain_detects_tampering	Audit	Chain valid; modified row detected at the right ID; valid again after restore
test_investigations_persisted	Persistence	Completed investigation has evidence and the "recommendation only" notice
test_create_order_uses_server_side_prices	Payment	PayPal status CREATED; server price used
test_create_order_idempotency_key_replays	Idempotency	Same key returns same order; no second PayPal order
test_cart_validation	Input validation	Unknown product, mixed sellers, quantity 99 rejected
test_staff_cannot_create_checkout	RBAC	403 for staff
test_capture_requires_buyer_approval	Payment state	409 when order not approved
test_low_risk_payment_is_captured_once	Capture, duplicates	LOW captured; replays never call PayPal capture again (exactly 1 call)
test_payment_ownership_enforced	Ownership	Another shopper gets 403
test_amount_mismatch_blocks_capture	Integrity	Tampered amount gives 409; no capture

Test	Area	Checks
test_high_risk_payment_cannot_bypass_review	Policy	HIGH held, APPROVED not captured, investigation started; direct capture refused (409)
test_reviewer_decision_controls	Review	Shopper 403, wrong password 401, wrong confirmation 422, short reason 422, escalation requires ADMIN, approve captures once, repeat 409
test_reject_decision_never_captures	Review	REJECT leaves PayPal order APPROVED, uncaptured
test_webhook_signature_verification_and_dedupe	Webhooks	Forged or missing signature 400; valid processed; duplicate ignored

15.2 Other verification performed

Check	Result
Frontend type check (`tsc -b`)	Passed
Frontend production build (`vite build`)	Passed
Manual UI walkthrough in a browser	All 9 console sections, the investigation workspace, storefront, checkout (unconfigured) and a reviewer decision exercised
PostgreSQL DDL generation (`alembic upgrade head --sql`)	18 tables generated for the PostgreSQL dialect
ML evaluation	Hold-out metrics in Part 7.6

15.3 Not yet verified

Item	Reason
Live PayPal Sandbox order, approval and capture	Sandbox credentials were not available on the build machine
Live webhooks	Requires a public URL and webhook ID
Groq LLM investigation	No API key; fallback path tested instead
Docker Compose and PostgreSQL at runtime	Docker daemon was not running on the build machine
Render deployment	Not deployed
Load and performance testing	Not performed
Frontend automated tests	None written

PART 17

Limitations and Future Scope

17.1 Current limitations (implemented system)

- **Synthetic training data.** Patterns were designed by the author; some features (for example hour of day) are strong because of how the generator works.
- **No production validation.** Live PayPal Sandbox and Groq paths are implemented but were not executed during development.
- **Uncalibrated probabilities.** Class weighting shifts scores; they are risk scores rather than calibrated probabilities.
- **Single-process design.** WebSocket fan-out, feature state and graph cache live in one process.
- **CAPTURE intent.** Held orders must be decided within PayPal's approval window.
- **Simplified context.** Device identity is a random browser ID; checkout location defaults to the customer's home city unless a demo provides one.
- **In-sample history scores.** Historical transactions are scored by a model that was trained on most of them; only hold-out metrics measure quality.
- **No frontend tests**, no load testing and no formal security review.

17.2 Future scope (not implemented)

Area	Proposed improvement
Data	Train and validate on real or public fraud datasets; time-based (out-of-time) validation.
Calibration	Isotonic or Platt calibration; cost-based threshold selection.
Monitoring	Feature drift, score drift and alerting; champion/challenger models.
Graph	Community detection (Louvain / Leiden), graph embeddings, graph neural networks, a graph database for scale.
Scale	Redis pub/sub or a message queue for events, a shared feature store, horizontal API scaling, streaming ingestion.
PayPal	AUTHORIZE-intent flow (authorise, then capture or void), refunds, disputes API, live webhook subscriptions, Vault.
Identity	Step-up verification for MEDIUM payments (one-time codes), stronger device intelligence.
Security	httpOnly cookie sessions, MFA for reviewers, penetration testing, secrets manager.
AI agent	Live Groq evaluation, prompt and report quality evaluation, multiple-model comparison.
Product	Case management, reviewer workload metrics, customer communication templates.